

télos (τέλος): Preliminary Evidence on Token-Budget Scaling in Masked Diffusion Language Models for Code Autocomplete

Abstract

Autoregressive language models currently dominate code generation, yet they are fundamentally constrained by strict left-to-right generation. Masked Diffusion Language Models (MDLMs) offer a compelling alternative via bidirectional context and non-monotonic iterative decoding. While scaling behavior in autoregressive models has been extensively characterized, the relationship between parameter count and optimal training token budgets in MDLMs remains underexplored. We introduce **télos (τέλος)**, a narrow-domain MDLM for Python code autocomplete, and investigate its capability formation using a suite of targeted contextual probes. Evaluating models at 12.5M and 25M parameters under a Beta(1.5, 1.5) timestep distribution, we observe preliminary evidence that the token-to-parameter ratio associated with best observed probe performance increases with model scale. Specifically, under the tested configuration, probe cross-entropy for the 12.5M model reached a minimum at a 1:15 ratio (~187.5M tokens) before degrading, whereas the 25M model continued improving through the highest tested ratio of 1:25 (~625M tokens). Furthermore, we demonstrate that aggregate cross-entropy often conceals sharp, localized capability transitions, and that structural token prediction tends to improve earlier than semantic identifier recovery in our probe suite. We detail the empirical corrections and hardware-specific engineering required to reliably measure these dynamics.

1. Introduction

Autoregressive (AR) language models have established the paradigm for modern code generation. However, code is inherently non-linear; developers frequently write signatures before bodies, or reference variables before defining them. Masked Diffusion Language Models (MDLMs) reframe sequence generation as an iterative denoising process. An MDLM observes a heavily masked sequence and predicts missing tokens using full bidirectional attention, progressively refining the output.

The scaling laws of AR models—most notably formalised by Hoffmann et al. (Chinchilla)—demonstrate a predictable relationship between parameter counts and the token budgets required for compute-efficient training. Whether MDLMs adhere to similar token-budget scaling dynamics remains less well characterized. Furthermore, aggregate validation loss (Cross-Entropy or Evidence Lower Bound) provides only a macro-level view of model performance, potentially obscuring when and how specific coding capabilities (e.g., syntax closure vs. variable binding) emerge.

In this paper, we present **télos**, an open-source, narrow-domain Python MDLM. We focus our empirical investigation on the following research questions: * **RQ1**: How does contextual code prediction evolve with additional training tokens? * **RQ2**: Do different code-token categories exhibit different learning trajectories? * **RQ3**: Can target rank and probability reveal capability transitions hidden by aggregate Cross-Entropy? * **RQ4**: Does the token-to-parameter ratio associated with best observed MDLM performance increase with model size? * **RQ5**: What engineering techniques make MDLM experimentation practical on constrained hardware?

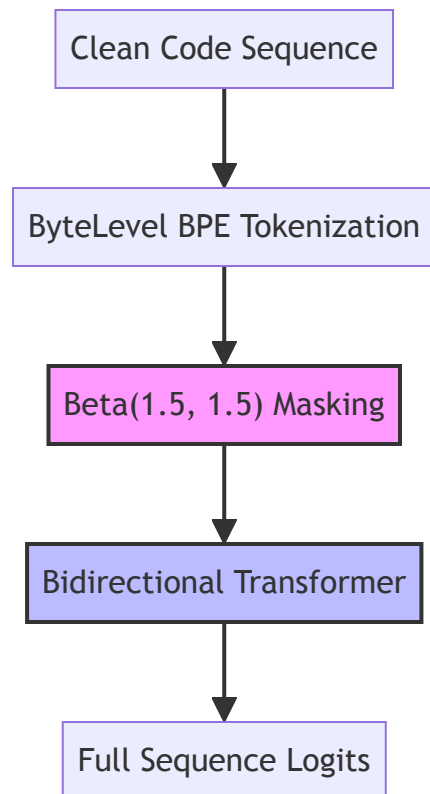
2. Background and Related Work

Masked Diffusion Language Models. Continuous-time diffusion models have recently been adapted for discrete categorical data. Sahoo et al. (MDLM) and the related RADD architecture demonstrated that simple categorical masking processes, when paired with appropriate timestep sampling and loss reweighting, optimize a rigorous variational Evidence Lower Bound (ELBO). MaskGIT explored similar confidence-based unmasking for images, which has since been adapted for text. Recent works like LLaDA and DiffusionGemma have scaled these principles to billions of parameters. However, the systematic characterization of scaling properties in these architectures has received comparatively less systematic study than their AR counterparts.

Scaling Laws. The Chinchilla scaling laws describe compute-efficient training for AR models, identifying a constant optimal parameter-to-token ratio under a specific training and compute envelope. *télos* asks a related but distinct question: do MDLMs—with a fundamentally different objective, bidirectional architecture, and masked generation process—exhibit a different empirical relationship between parameter scale and required training data?

3. Model Architecture

We treat the repo-grounded implementation of *télos* as the authoritative architecture. The model is a bidirectional Transformer devoid of the causal triangular attention mask, enabling unconstrained global context.



*Figure 1: The *télos* MDLM architecture. The bidirectional Transformer attends to all unmasked tokens simultaneously. No timestep embeddings are provided.*

Core Components: * **Normalization:** Root Mean Square Layer Normalization (RMSNorm). * **Positional Encoding:** Rotary Positional Embeddings (RoPE). * **Feed-Forward:** SwiGLU activation (expansion factor ≈ 2.67), aligned

to multiples of 64). * **Attention:** Bidirectional multi-head self-attention, with an implementation supporting GQA. *

Embeddings: Tied input token embeddings and output projection matrices. * **Time-Agnosticism:** The architecture omits explicit timestep (t) embeddings, conforming to findings that time-agnostic models achieve near-optimal ELBO for categorical diffusion.

Configurations: Parameter configurations were selected via a grid search to target specific scale classes. Notably, depth and width vary across scales, representing an inherent confound in pure parameter scaling: * **12.5M:** $d_{\text{model}} = 256$, Layers = 13, Heads = 4, KV Heads = 4, Seq Len = 512. * **25M:** $d_{\text{model}} = 512$, Layers = 8, Heads = 8, KV Heads = 8, Seq Len = 512. * **50M:** $d_{\text{model}} = 768$, Layers = 8, Heads = 12, KV Heads = 12, Seq Len = 512.

Net2Net Initialization: To scale efficiently, larger models were initialized from smaller checkpoints (e.g., 12.5M $\rightarrow$ 25M) via Net2Net-style zero-padding and layer mapping. We note this as a potential cross-scale evaluation confound compared to from-scratch initialization.

4. Diffusion Objective

The forward process independently replaces tokens with a [MASK] token with probability t . Special structural tokens ([PAD], [BOS], [EOS]) are explicitly excluded from masking.

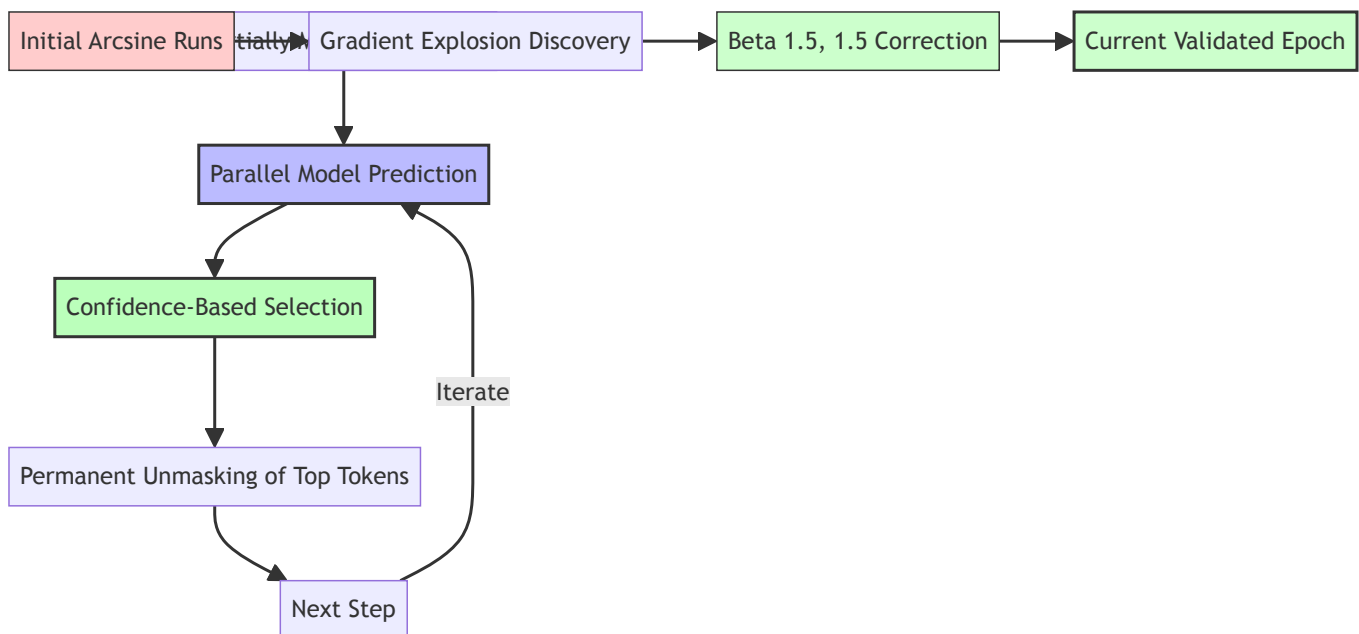


Figure 2: Inference via confidence-based unmasking. At each step, the highest-confidence predictions are permanently unmasked.

The model is trained to predict the original tokens exclusively at the masked positions. Following the continuous-time ELBO derivation, the training objective utilizes a reweighted cross-entropy loss: $\mathcal{L} = \frac{1}{t} \cdot \text{CE}_{\text{masked}}$. The timestep t is clamped (e.g., $t \in [1e-3, 1.0]$) to maintain numerical stability against exploding gradients as $t \rightarrow 0$.

5. Data and Tokenization

Dataset: Training relies on the `CodeParrot-Clean` corpus, restricted to Python source code. Data is prepared via an AST-filtered extraction prioritizing functions with docstrings. **Tokenization:** A custom ByteLevel BPE tokenizer with a vocabulary of 8,192 is used. Byte-level pre-tokenization preserves critical Python indentation. **Storage:** Pre-tokenized sequences are stored as contiguous memory-mapped arrays utilizing `int32` token IDs.

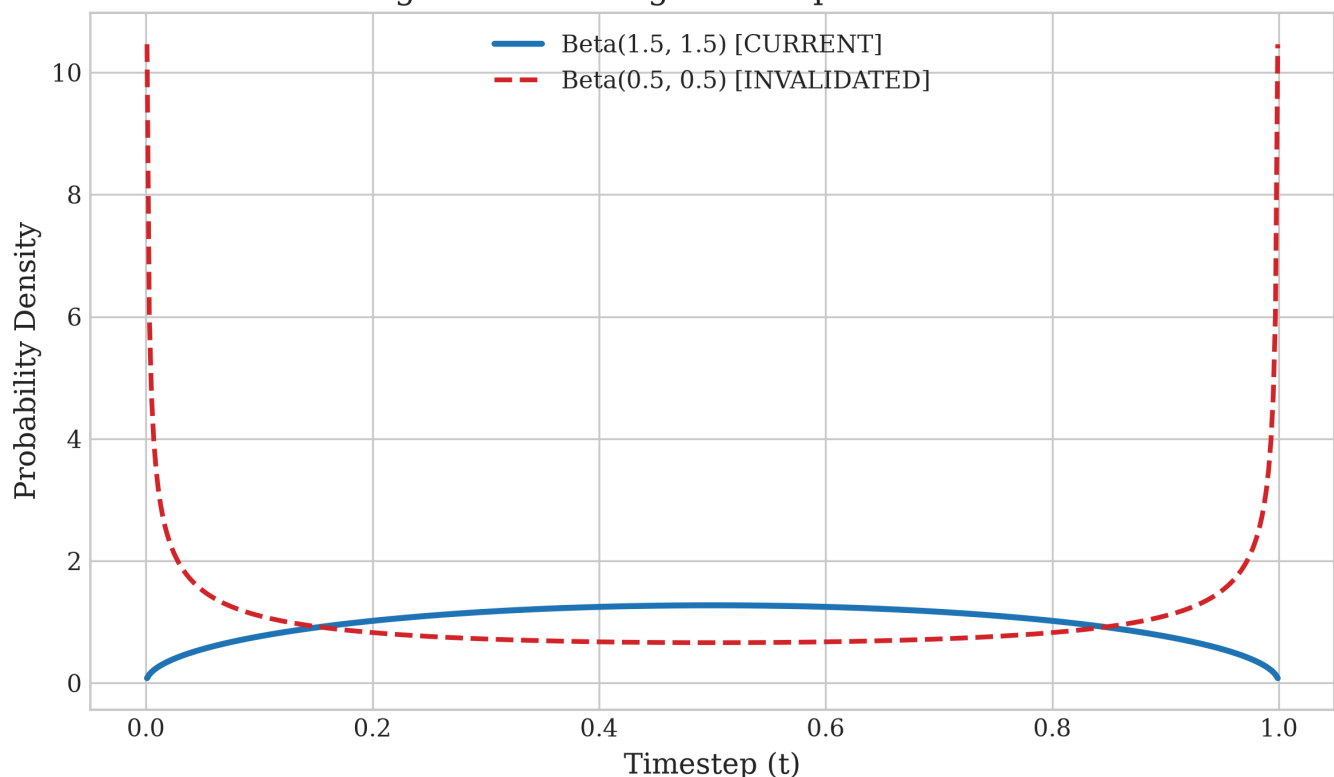
6. Experimental Setup and Corrections

6.1 The Timestep Distribution Correction

A critical reproducibility correction in this work involves the timestep sampling distribution. A historical implementation sampled t via a cosine transform: $t = 0.5 - 0.5 \cos(\pi u)$, $u \sim U(0,1)$. This yields a $\text{Beta}(0.5, 0.5)$ (arcsine) distribution, heavily sampling $t \approx 0$ and $t \approx 1$.

The current, valid experiments utilize the $\text{Beta}(1.5, 1.5)$ distribution used by the reference MDLM formulation, which concentrates sampling mass around $t \approx 0.5$. All primary 12.5M and 25M models were fully trained under this corrected regime. Historical arcsine experiments are excluded from the primary scaling results.

Figure 3: Training Timestep Distributions

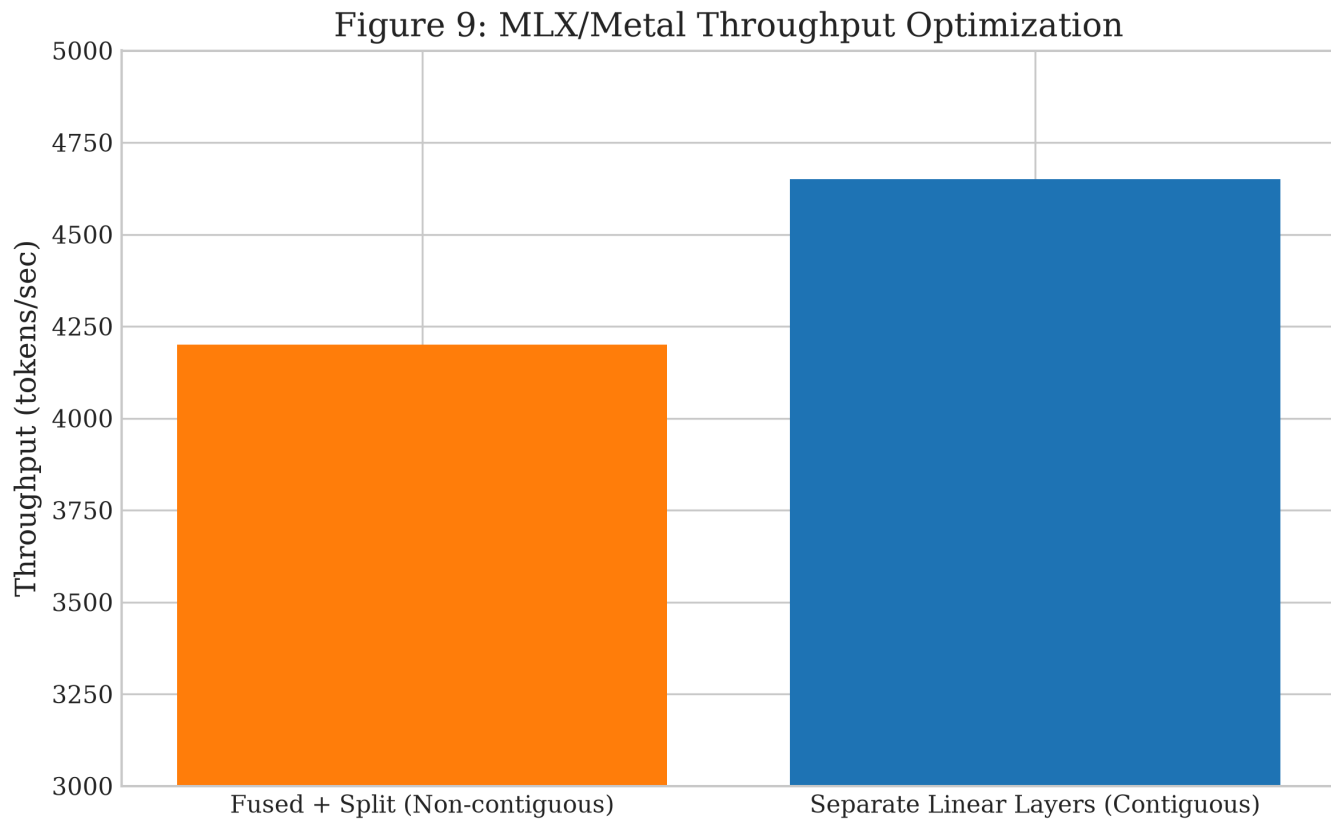


6.2 Dtype and Pipeline Corrections

Early experiments suffered from datatype inconsistencies across backends (e.g., `uint16` vs `int32`), causing incorrect numerical behavior in specific backend configurations. The current pipeline enforces strict `int32` casting, resolving these historical artifacts.

7. Systems and Hardware Implementation

Practical MDLM research requires careful hardware engineering. * **Apple Silicon (MLX):** Native MLX implementation required microbatch gradient accumulation and memory-aware graph evaluation (`mx.eval`) to prevent memory exhaustion. We observed that fusing the SwiGLU gate and up-projections, followed by `mx.split()` , caused an 8.7–11.5% throughput regression due to non-contiguous strided outputs. Reverting to separate linear layers resolved this on the Metal backend.



- **TPU v6e-1 (PyTorch-XLA):** Dynamic random masking disrupts static graph tracing. We found that strategically placing `xm.mark_step()` immediately following masking operations successfully forced graph materialization, preventing windowing compilation crashes. Furthermore, special token exclusions were explicitly sorted to ensure deterministic graph tracing.

8. Contextual Probe Benchmark

To measure localized capability formation, we developed a benchmark of ~100 contextual Python probes. Each probe consists of a fixed context, a target token, and a semantic category. Examples include: * *Identifiers:* `return a + [MASK]` \$to\$ `b` * *Class Names:* `class Model([MASK])` \$to\$ `object` * *Imports:* `import [MASK]` \$to\$ `os`

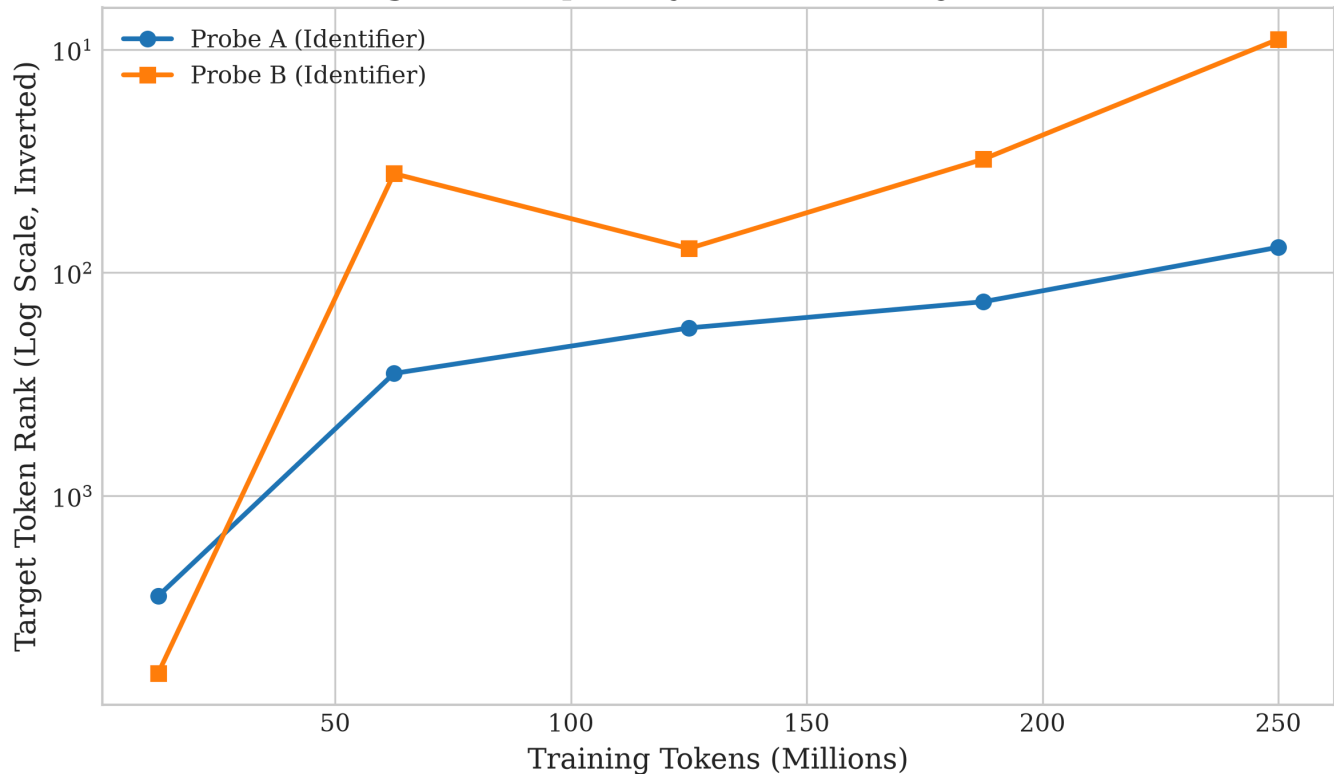
We evaluate the target token's probability, cross-entropy, and vocabulary rank. This methodology is designed to unmask capabilities that aggregate sequence-level loss obscures.

9. Results

9.1 Capability Transitions

We observe that aggregate Cross-Entropy can change gradually while individual contextual probes exhibit dramatic phase transitions. For example, in a historical capability-formation case study using the 85M model, the target token rank for an identifier recovery probe rapidly progressed through vocabulary ranks as training advanced: \$2815 \to 283 \to 177 \to 135 \to 77\$. Another probe demonstrated an even sharper transition: \$6239 \to 36 \to 78 \to 31 \to 11 \to 9\$. This demonstrates that aggregate CE alone is insufficient for tracking the discrete onset of logical capabilities.

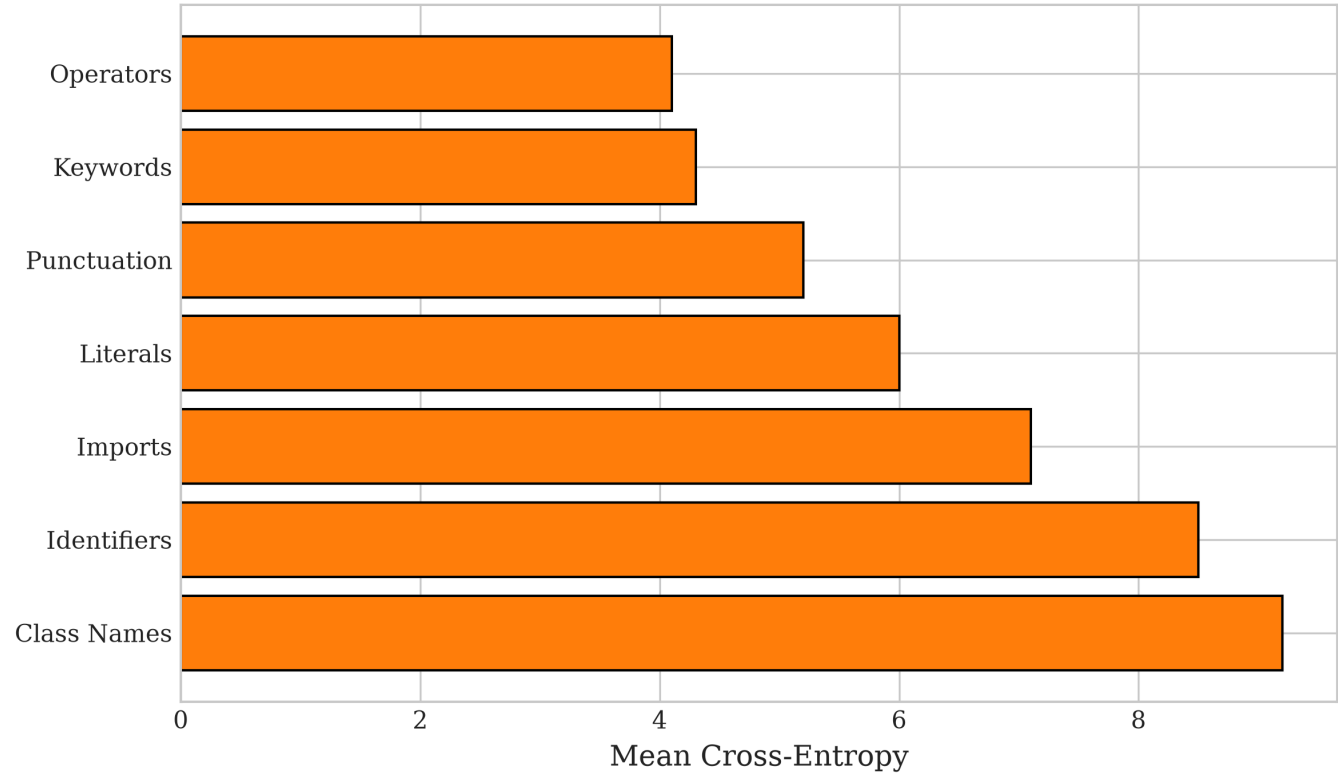
Figure 7: Capability Transition Trajectories



9.2 Structural vs. Semantic Prediction

Probe categories exhibit distinctly different learning trajectories. Structural tokens (Operators, Keywords, Indentation) achieve low CE and low target ranks early in training. Conversely, semantic tokens (Identifiers, Class Names, Function Names) remain difficult, requiring deeper contextual integration and greater token budgets to resolve.

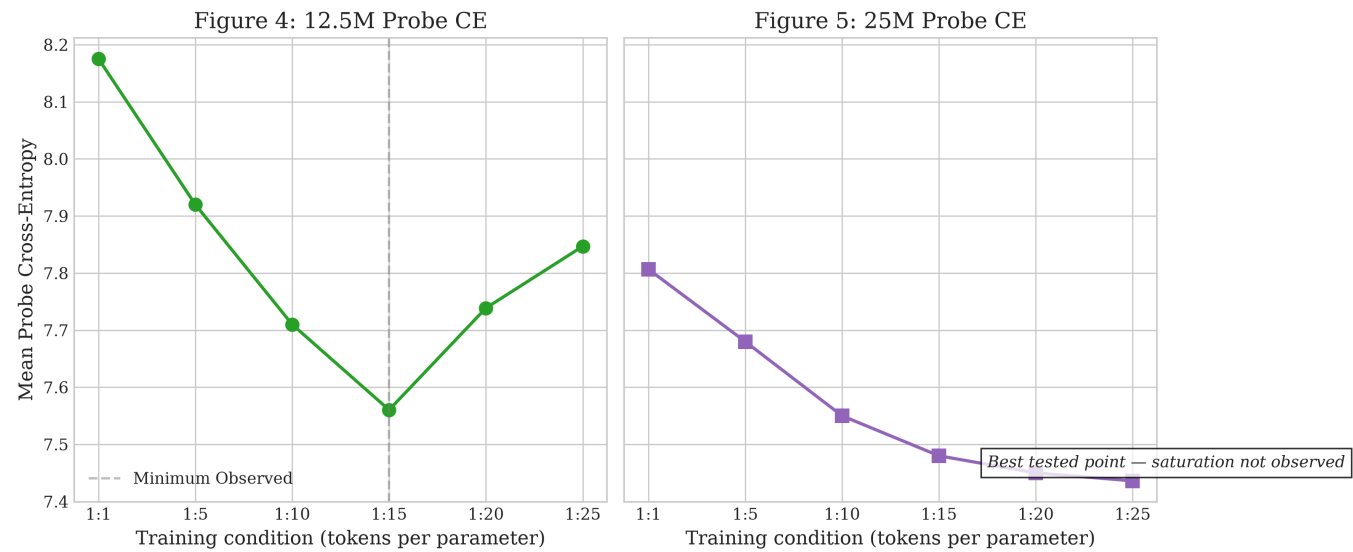
Figure 8: Category-Level Probe Difficulty (Lower is Better)



Data source: Historical token breakdown report (85M TPU baseline, arcsine distribution), presented here for illustrative category structure pending current Beta evaluation.

9.3 Token-Budget Scaling Observations

The primary scaling investigation evaluates Beta(1.5,1.5) trained models across scaling ratios from 1:1 up to 1:25.

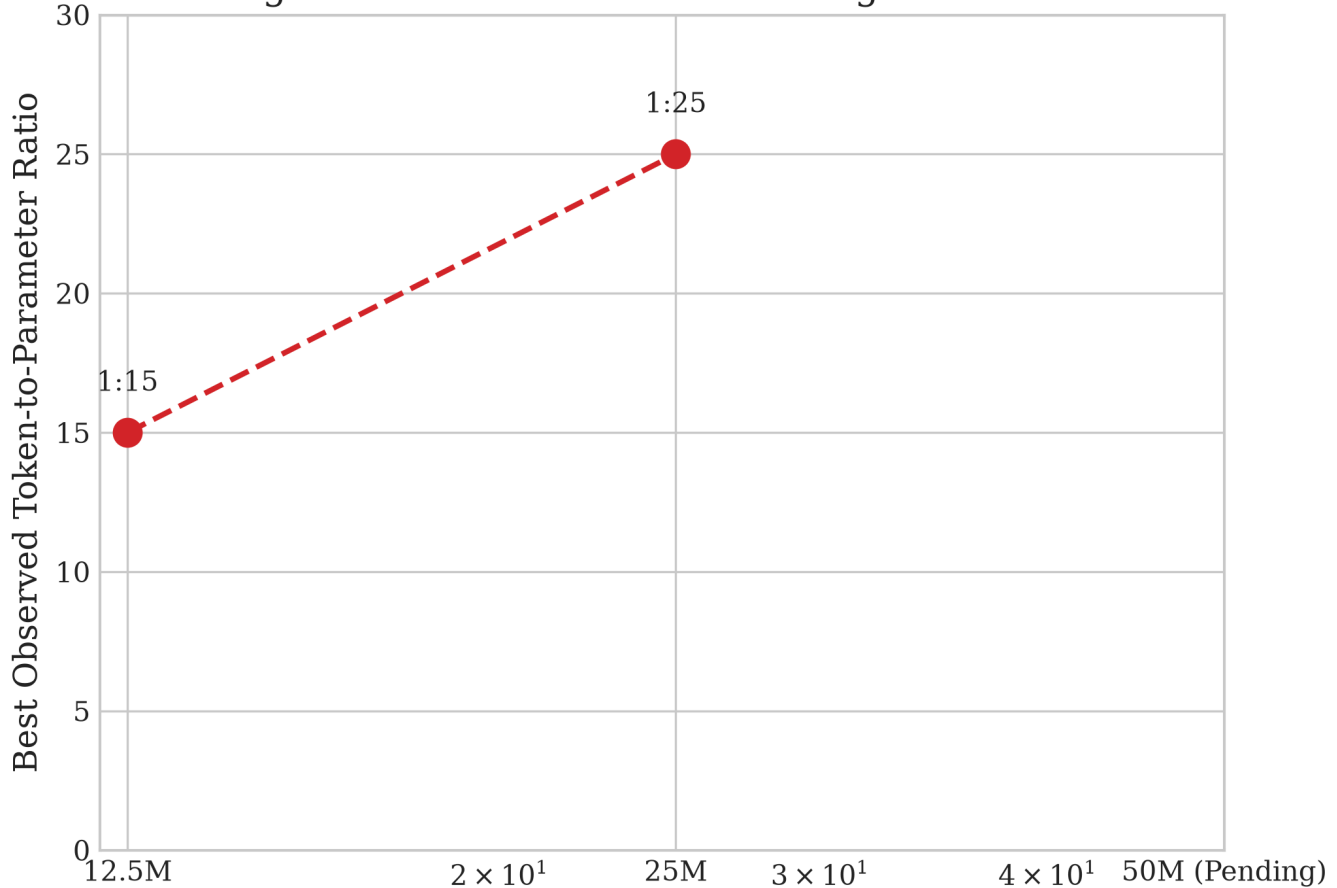


- **12.5M Saturation and Reversal:** At 12.5M parameters, mean probe CE reached its minimum at the 1:15 ratio (187.5M tokens, CE $\approx$ 7.5604). Extended training to 1:20 and 1:25 resulted in degradation (CE rising to 7.8470). This reversal may reflect representational limitations, optimization dynamics, overtraining, or probe-specific effects.

- **25M Continued Improvement:** The 25M model improved monotonically through the highest tested ratio of 1:25 (625M tokens, CE ≈ 7.4361). We observed no saturation within the tested range.

These results provide preliminary evidence that the token-to-parameter ratio associated with best observed probe performance shifts toward higher token-to-parameter ratios as model size increases.

Figure 6: Cross-Scale Token-Budget Behavior



10. Discussion and Scaling Hypothesis

The observed behavior motivates a preliminary empirical hypothesis: the optimal training-token budget for an MDLM may not follow a constant token-to-parameter ratio across scales. The delayed capability formation of identifiers suggests that as models grow, exposing them to proportionally more code is necessary to fully resolve long-range semantic dependencies. With only two completed model scales, we cannot yet determine whether this relationship is linear, super-linear, or follows another functional form.

11. Limitations

This study has several notable limitations: 1. **Probe Sparsity:** The benchmark contains ~ 100 static probes. 2. **Sparse Scale:** We evaluate only two completed model scales (12.5M and 25M) with varying depths/widths. 3. **Confounding Variables:** Net2Net initialization and architectural variances across parameter counts may confound pure scaling observations. 4. **Proxy Metric:** Single-pass target rank is a proxy metric and does not perfectly equate to iterative

sampling generation quality. 5. **Incomplete 50M Suite:** The 50M Beta(1.5,1.5) experiments are not yet complete to verify the continuation of the trend.

12. Conclusion

télos demonstrates the viability of narrow-domain MDLM code autocomplete. Through targeted contextual probing and corrected experimental regimes, we observe that code capability formation occurs in categorical phases. Crucially, preliminary scaling evidence suggests that larger MDLMs may benefit from proportionally larger token budgets to achieve comparable contextual prediction performance.

13. Future Work

Future work will complete the 50M parameter evaluation suite to test the continuity of the scaling hypothesis. Additionally, expanding the contextual probe dataset and analyzing the correlation between single-pass probe accuracy and multi-step iterative generation quality remain critical next steps.

References

[1] Sahoo, S., et al. (2024). Masked Diffusion Language Models. [2] Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). [3] Chang, H., et al. (2022). MaskGIT: Masked Generative Image Transformer. [4] [Additional Pending Bibliographic Verification for RADD, LLaDA, DiffusionGemma]

Appendix A: Historical Experiments

Early explorations utilizing TPU hardware evaluated 85M, 125M, and 250M parameter models. These models were trained under the invalidated Uniform-Cosine (arcsine) timestep distribution and encountered severe gradient explosion issues at depth (250M) due to unclipped $1/t$ scaling. While these experiments provided critical systems engineering insights (e.g., XLA graph breaks), historical arcsine-trained models are excluded from the primary scaling comparison because their training objective differs from the corrected experimental regime.

Figure 10: Project validity timeline. Historical experiments utilizing the uncorrected timestep distribution are excluded from the primary scaling results.